



Departamentul Automatică și Informatică Industrială

**Facultatea Automatică și Calculatoare
Universitatea Națională de Știință și Tehnologie
POLITEHNICA din București**

Splaiul Independenței 313, 060042, București, România
Sala ED 412, Tel. 021/402.92.69
www.aii.pub.ro, email: secretariat.aii@upb.ro



Sesiunea de Comunicări Științifice Studențești

Ediția 2025

Sistem pentru detecția și raportarea evenimentelor de securitate în rețele industriale de comunicații

**Autor - Theodor Predescu, Automatică și Calculatoare, specializarea Ingineria
Sistemelor, anul VI, grupa 342 A1**

Adresă mail - predescutheodor@gmail.com

Conducător științific: - Conf. dr. Ing. Florin Daniel Anton

Cuprins:

Introducere.....	3
Prezentarea domeniului.....	4
Descrierea problemei și soluția propusă.....	4
Prezentarea aplicației.....	7
Detalii de implementare.....	7
Componenta de intermediere.....	8
Componenta centrală.....	9
Componenta de administrare	9
Experiența utilizatorului	13
Concluzii și planuri de viitor	17
Bibliografie.....	18

Introducere

Tehnologia a devenit în ultimele decenii unul dintre principalii factori care modelează societatea modernă, generând în mod constant noi oportunități de inovație și dezvoltare în domeniul sistemelor software și hardware. Domenii precum robotica, automatizarea, dezvoltarea aplicațiilor software și a jocurilor video au evoluat rapid și au devenit piloni esențiali în societatea modernă. De la jocuri revoluționare, precum *Doom*, până la sisteme complexe de operare, precum *Linux*, numeroase proiecte demonstrează impactul semnificativ pe care îl pot avea inițiativele individuale sau colaborative asupra dezvoltării tehnologice.

Odată cu această dezvoltare, a apărut însă și o nevoie sporită de protejare împotriva factorilor externi ce pot perturba fluxul de lucru, problemă cu atât mai amplificată în domeniile cu un număr mare de componente software, precum domenii industriale.

Într-un studiu al protocoalelor folosite în mediul industrial — Printre aceste protocoale analizate se numără Modbus, Siemens S7, EthernetIP, BACnet, OMRON FINS, Codesys — s-a constatat faptul că acestea au fost proiectate pentru medii izolate, fără a include mecanisme de securitate precum criptarea sau autentificarea. [10]

Analiza traficului industrial de control (ICS) arată o creștere semnificativă a expunerii echipamentelor tehnice la Internet. Studii bazate pe date din „*CAIDA/UCSD network telescope*” arată faptul că traficul asociat protocolului *Modbus* (TCP/502) nu a înregistrat variații semnificative până la începutul anului 2014, dar ulterior a fost observată o creștere bruscă de aproximativ trei ordine de mărime în numărul de adrese IP scanate și aproximativ un ordin de mărime în numărul de surse ce scanează activ Internetul.

Creșterea aceasta este corelată cu apariția și popularizarea instrumentelor de scanare la scară largă, precum și interesul crescut al comunității de cercetare și expunerea publică a infrastructurilor publice. Acestea evidențiază faptul că ecosistemul ICS devine din ce în ce mai expus în mediul Internet global, ceea ce impune necesitatea unor metode riguroase de identificare și filtrare a traficului ICS.

Aplicația prezentată în această lucrare propune o soluție de securizare împotriva acestor factori externi, prin utilizarea unui sistem de control bazat pe politici. Soluția permite monitorizarea și filtrarea traficului dintre echipamente, oferind posibilitatea definirii clare a

conexiunilor permise. Această abordare este destinată în special mediilor industriale, unde relațiile de comunicare dintre componente sunt bine definite și necesită un nivel ridicat de control și predictibilitate.

Prezentarea domeniului

Pe măsura creșterii în complexitate a ecosistemului digital aflat în continuă expansiune, securitatea aplicațiilor și a serviciilor devine tot mai dificilă și mai importantă. Pentru a funcționa în mod eficient și a fi utilizate la capacitate maximă, aceste servicii necesită un nivel ridicat de securitate, capabil să protejeze împotriva numeroaselor tipuri de atacuri. Serviciile de securitate sunt supuse unor cerințe în continuă creștere, dorindu-se adaptarea atât la complexitatea amenințărilor, cât și la volumul tot mai mare de trafic generat de creșterea numărului de utilizatori și a dispozitivelor conectate.

În mod particular, mediile industriale impun cerințe suplimentare de securitate, datorită infrastructurilor critice și necesității funcționării stabile. În aceste scenarii, schimbul de date dintre echipamente este strict regulat și supus unui nivel de control ridicat, pentru a preveni degradarea performanței sau chiar oprirea funcționării echipamentelor. Din acest motiv, soluțiile de securitate destinate mediilor industriale trebuie să ofere mecanisme eficiente de monitorizare, filtrare și control al comunicațiilor dintre dispozitive.

Un serviciu de securitate are rolul de a proteja sistemele împotriva acțiunilor nedorite și a actorilor malițioși, prin identificarea și remedierea vulnerabilităților existente. În acest context, mecanismele de protecție trebuie să funcționeze în mod riguros și continuu, acoperind un spectru cât mai larg de puncte slabe. În sens contrar, compromiterea sistemelor poate fi realizată prin exploatarea unei singure vulnerabilități neidentificate sau tratate insuficient. Această realitate evidențiază necesitatea menținerii unui standard ridicat pentru mecanismele de securizare, ceea ce impune dezvoltarea unor soluții eficiente, adaptabile și capabile să răspundă în timp real la evoluția amenințărilor.

Descrierea problemei și soluția propusă

O cerință esențială în mediul industrial reprezintă monitorizarea dispozitivelor în cadrul unei rețele locale. Se dorește un nivel cât mai mare de control asupra conexiunilor ce pot avea loc între dispozitive.

Monitorizarea dispozitivelor în cadrul unei rețele implică analiza detaliată a pachetelor de date și identificarea comportamentelor anormale sau care nu se conformează cu politicile de comunicare stabilite.

Lucrările recente din domeniul securității sistemelor industriale se concentrează predominant pe detectarea anomaliilor prin metode statistice și de învățare automată. *Jonguk Kim, Jeong-Han Yun și Hyoungho Chun Kim* [4] propun o metodă de detecție bazată pe rețele neuronale de tip *sequence-to-sequence*, antrenate pe date operaționale normale, care identifică abateri prin analiza erorilor de predicție. *Luyao Wang et al.* [5] extinde această direcție, prezentând tehnici de *transfer learning* și *clustering*, adresând problema variației distribuției traficului între scenarii diferite. Deși eficiente în detectarea comportamentelor anormale, ambele abordări sunt în esență reactive și dependente de seturi de date reprezentative, informații ce se pot dovedi dificil de obținut în medii industriale reale. Lucrarea de față adoptă o perspectivă diferită asupra problemei. În locul unui model care învață să distingă traficul normal de cel anormal, soluția propusă se bazează pe un mecanism de control explicit, în care conexiunile permise sunt definite în mod direct de administratorul sistemului. Filtrarea traficului se realizează la nivel kernel, oferind o viteză de răspuns mult mai mare și un grad ridicat de control asupra comunicațiilor din rețea. Această abordare elimină dependența de date de antrenament și reduce ambiguitatea specifică modelelor probabilistice, oferind în același timp moduri de funcționare flexibile care permit adaptarea dinamică a politicilor de acces în funcție de cerințele operaționale. Componenta designată echipamentelor monitorizate utilizează mecanisme de inspecție și filtrare la nivel *kernel* specifice sistemelor de operare de tip *Linux*. Soluția presupune interceptarea și preluarea informațiilor de interes a pachetelor de date în puncte critice ale fluxului acestora, urmată de o filtrare eficientă și rapidă. Informații despre pachetele analizate sunt apoi transmise unei unități centrale pentru o inspecție generală.

Filtrarea acestora se realizează pe baza unei liste definite de utilizator, care specifică atât adresele autorizate pentru comunicare, cât și porturile pe care acestea le pot folosi.

Modurile de funcționare ale aplicației sunt reprezentate de patru profile disponibile:

- **LISTENING** – aplicația rulează în mod permisiv, acceptând conexiuni nepermise în listă, dar acestea sunt raportate
- **REACTIVE** – rulează în mod strict, refuzând conexiunile ce nu sunt permise; conexiunile respinse sunt raportate mai departe
- **DISABLED** – aplicația permite conexiuni ce nu sunt definite, iar raportarea evenimentelor este dezactivată

- **MONITORING** – aplicația este în procesul de adăugare automată a conexiunilor în lista de conexiuni permise (acest mod de funcționare este folosit pentru definirea accesului în mod dinamic).

Soluția propusă cuprinde o aplicație desktop concepută pentru administrarea sistemului. Pe baza raportărilor făcute, administratorul sistemului poate avea o imagine de ansamblu a traficului produs în rețea și a regulilor de funcționare a fiecărui dispozitiv. Aplicația face ușor de depistat dispozitivele ce încep o comunicare ce se abate de la profilele setate în rețea, indiferent dacă comunicarea a fost blocată sau permisă. De asemenea, aplicația facilitează modificarea dinamică a profilelor dispozitivelor din rețea, atât a modului de lucru cât și a modificării dispozitivelor permise.

Prezentarea aplicației

Aplicația utilizează o arhitectură modulară, compusă din componente cu responsabilități bine delimitate. Aceste componente sunt:

1. **componenta de filtrare la nivel kernel**, responsabil de interceptarea pachetelor, extragerea informațiilor relevante, transmiterea acestora pentru o inspecție generală și blocarea pachetelor definite în fișierul de configurație.
2. **componenta de intermediere**, responsabil de intermedierea informațiilor între modulul kernel și componenta centrală. Acesta de asemenea este de asemenea responsabilă de citirea fișierelor de configurații; sincronizarea acestora cu informațiile primite de la server și trimiterea noilor dispozitive ce trebuiesc blocate către modul.
3. **componenta centrală**, responsabilă de stocarea tuturor informațiilor primite de la dispozitivele monitorizate și intermedierea legăturii dintre totalitatea dispozitivelor și aplicația de administrare.
4. **componenta de administrare**, ce oferă posibilitatea de vizualizare a conexiunilor ce au avut loc pe dispozitivele monitorizare și modificarea configurațiilor acestora la nivel individual sau global.

Informațiile trimise, alături de configurarea curentă a echipamentelor, sunt stocate în într-o bază de date *SQLite*, pentru a oferi flexibilitate și posibilitatea analizării în detaliu a conexiunilor anterioare, precum și a modificărilor efectuate asupra configurației. Acest proces este gestionat de către *componenta centrală*.

Detalii de implementare

Aplicația este organizată în trei directoare principale, fiecare corespunzând unei componente distincte sistemului. Primul director conține componentele care rulează extern echipamentului monitorizat, al doilea include logica **componentei de intermediere** (*daemon*), iar ultimul este destinat **componentei de filtrare la nivel kernel**.

În directorul asociat componentelor externe echipamentului sunt incluse atât **componenta de administrare** (aplicația *desktop*), cât și **componenta centrală** responsabilă de gestionarea și coordonarea sistemului [Fragment de cod 1. Directorul de lucru].

```
./daemon
./document
./external
./kernel_module

./external/admin_application/
./external/server/
```

Fragment de cod 1. Directorul de lucru

Componenta de filtrare la nivel kernel

Această componentă reprezintă un modul kernel scris în limbajul de programare **C** ce rulează în spațiul privilegiat. Principala caracteristică a acestuia constă în definirea și înregistrarea unui mecanism de interceptare a pachetelor (*network hook*) utilizând cadrul software „**Netfilter**”, care permite interceptarea, analiza și blocarea pachetelor în rețea, în anumite puncte specifice ale procesului de procesare. Cu ajutorul acestui cadrul software, este înregistrată pentru reapelare funcția „*packet_hook*” la intrarea pachetelor în procesarea locală („*NF_INET_LOCAL_IN*”). Funcția este responsabilă de filtrarea pachetelor ce au ca și destinație echipamentul (în modul de funcționare REACTIVE), și raportarea informațiilor despre pachetele suport.

Componenta de intermediere

Această componentă reprezintă un agent local, „*daemon*”, ce rulează în spațiul utilizatorului. Limbajul ales pentru crearea acestei componente a fost **C++**, pentru a utiliza componente cu un grad de abstractizare mai mare, păstrând totodată o putere și viteză ridicată de procesare. Agentul este responsabil de trimiterea informațiilor extrase de *componenta de filtrare* spre *unitatea centrală* și de sincronizarea setărilor și regulilor noi selectate *componenta de administrare* cu regulile și modul de funcționare a *componentei de filtrare*.

Programul comunică cu modulul kernel prin intermediul unui „**Miscellaneous character device**”, care oferă posibilitatea folosirii unui *character driver* într-un mod simplificat. Modulul *kernel* trimite informații referitoare la pachetele recepționate într-un *character device*, informații ce sunt periodic citite de agentul local și trimise, în grupuri, către *componenta centrală*.

Într-un mod similar, programul trimite periodic mesaje către server pentru a verifica dacă configurarea aplicației trebuie modificată. În cazul în care serverul confirmă că au avut loc modificări, acesta va transmite noua configurare modulul kernel.

De asemenea, componenta este responsabilă de stocarea locală a configurației primite de la server, cat și de citirea acesteia la începutul rulării programului. Această stocare se face în trei fișiere, responsabile de reținerea unui identificator unic oferit dispozitivului, salvarea modului de rulare a aplicației, a locației fișierului ce conține lista de conexiuni permise și informații despre conexiunea cu serverul (adresa ip și portul ce trebuie utilizat), și fișierul responsabil de reținerea dispozitivelor ce trebuiesc blocate.

Componenta centrală

Componenta centrală reprezintă serverul aplicației, responsabilă de gestionarea legăturii dintre echipamentele utilizate și *componenta de administrare*. Aceasta centralizează informațiile transmise de *componentele de intermediere* și reformatează datele pentru fi utilizate de *componenta de administrare*. Totodată, prin componenta centrală, noile reguli setate din aplicația desktop sunt transmise către dispozitive.

Componenta de administrare

Componenta de administrare este reprezentată de aplicația desktop ce oferă flexibilitate în modificarea configurațiilor echipamentelor. Prin aceasta, administratorul rețelei poate modifica atât modul de rulare, cat și regulile de acces a conexiunilor. De asemenea, aplicația oferă posibilitatea de vizualizare sub forma unui graf a dispozitivelor și a traficului de date a acestora.

Această funcționalitate este creată cu ajutorul utilitarului *open-source* specializat vizualizărilor grafice **Graphviz**, prin intermediul algoritmului „*fdp*” (Force-Directed Placement), destinat generării automate a grafurilor pe baza relațiilor dintre noduri. Conexiunile dintre echipamente sunt transformate într-un fișier temporar în format DOT, format standard utilizat de **Graphviz** pentru descrierea grafurilor. Ulterior, aplicația execută procesul de generare a pozițiilor nodurilor și citește coordonatele rezultate pentru afișarea topologiei în interfața grafică [Fragment de cod 2. Generare graf]. Utilizarea algoritmului „*fdp*” permite distribuția automată și echilibrată a nodurilor, reducând suprapunerile și îmbunătățind lizibilitatea reprezentării grafice.

```

    // Preluare locația fișierelor temporale
    std::string temp_dir;
#ifdef _WIN32
    char temp_path[MAX_PATH];
    GetTempPathA(MAX_PATH, temp_path);
    temp_dir = temp_path;
#elif __linux__
    temp_dir = "/tmp/";
#endif

    std::string dot_file_path = temp_dir + "graph.dot";
    std::string txt_file_path = temp_dir + "graph.txt";

    // Generare dot_file
    std::string dot = "graph G {\n";
    dot += "node [shape=box];\n";
    dot += "graph [overlap=false, sep=\"1\", K=0.1];\n";

    ...

#ifdef _WIN32
    std::string cmd = "fdp -Tplain -Gstart=2 \"" + dot_file_path + "\" -o \"" + txt_file_path
+ "\"";
#else
    std::string cmd = "fdp -Tplain -Gstart=2 " + dot_file_path + " -o " + txt_file_path + "
2>/dev/null";
#endif
    system(cmd.c_str());

```

Fragment de cod 2. Generare graf

Pentru asigurarea persistenței informațiilor primite, odată cu ajungerea datelor în *componenta centrală*, acestea sunt salvate pe disc. Acest proces este făcut cu ajutorul unei baze de date **SQLite**. La pornire, aplicația verifică existența în directorul curent a bazei de date cu numele „*network_logs.db*”, iar în cazul în care aceasta nu este găsită, este creată automat, împreună cu cele două tabele SQL aferente [Fragment de cod 3. Configurare bază de date].

```

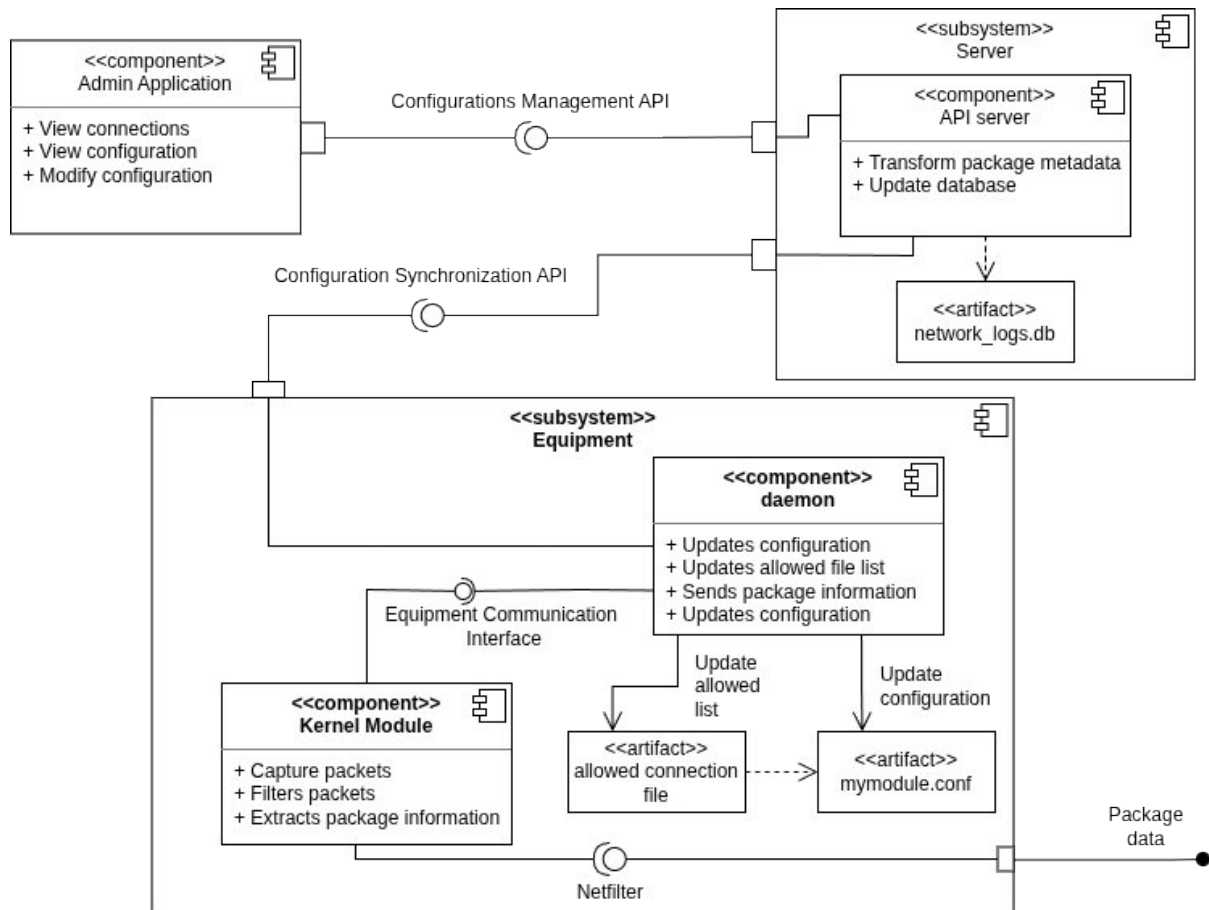
CREATE TABLE IF NOT EXISTS connections (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    mode INTEGER,
    pc_id TEXT NOT NULL,
    ts TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    src_ip TEXT NOT NULL,
    dst_ip TEXT NOT NULL,
    src_port INTEGER,
    dst_port INTEGER,
    protocol INTEGER,
    ttl INTEGER,
    packet_len INTEGER,
    iface TEXT,
    src_mac TEXT,
    dst_mac TEXT,
    tcp_flags INTEGER
);

CREATE TABLE IF NOT EXISTS pc_configs (
    pc_id TEXT PRIMARY KEY,
    config_file TEXT,
    allowed_file TEXT,
    icon BLOB,
    name TEXT DEFAULT "",
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

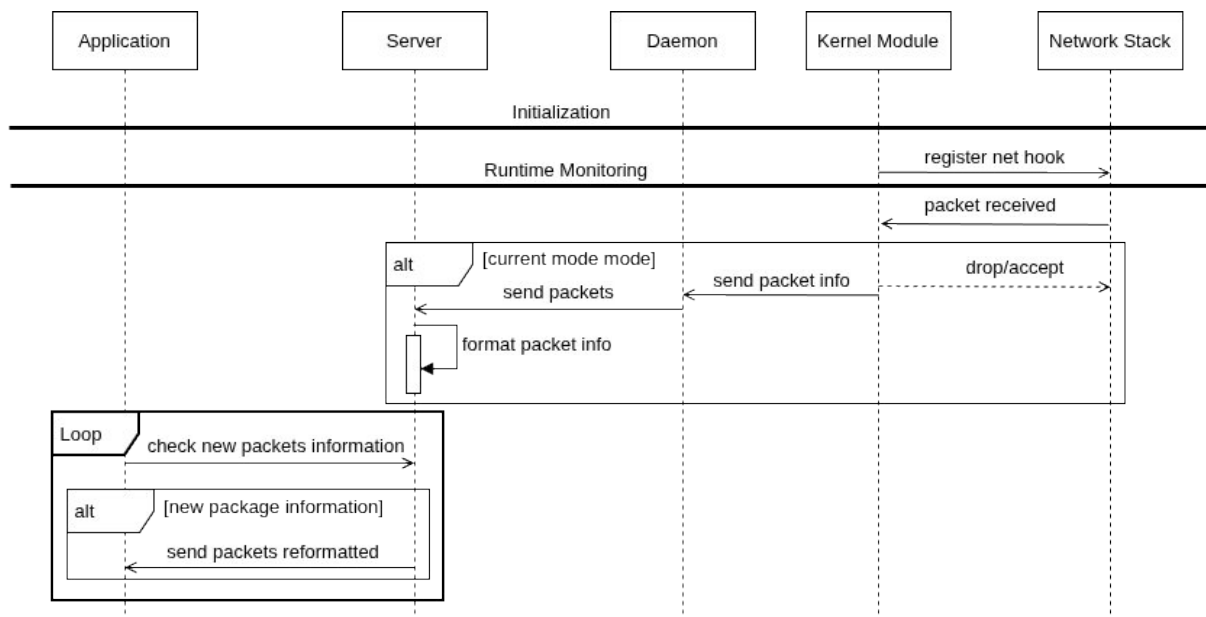
Fragment de cod 3. Configurare bază de date

Aplicația este compusă din patru componente logice distribuite pe trei medii de execuție distincte: *componenta de filtrare* și *componenta de intermediere* rulează la nivelul echipamentului ce va fi monitorizat, *componenta centrală* care funcționează ca server, iar *componenta de administrare*, care reprezintă interfața utilizată pentru configurarea și monitorizarea sistemului. Relația dintre aceste medii de execuție este descrisă de diagrama de componente [Imagine 1. Diagramă de componente].

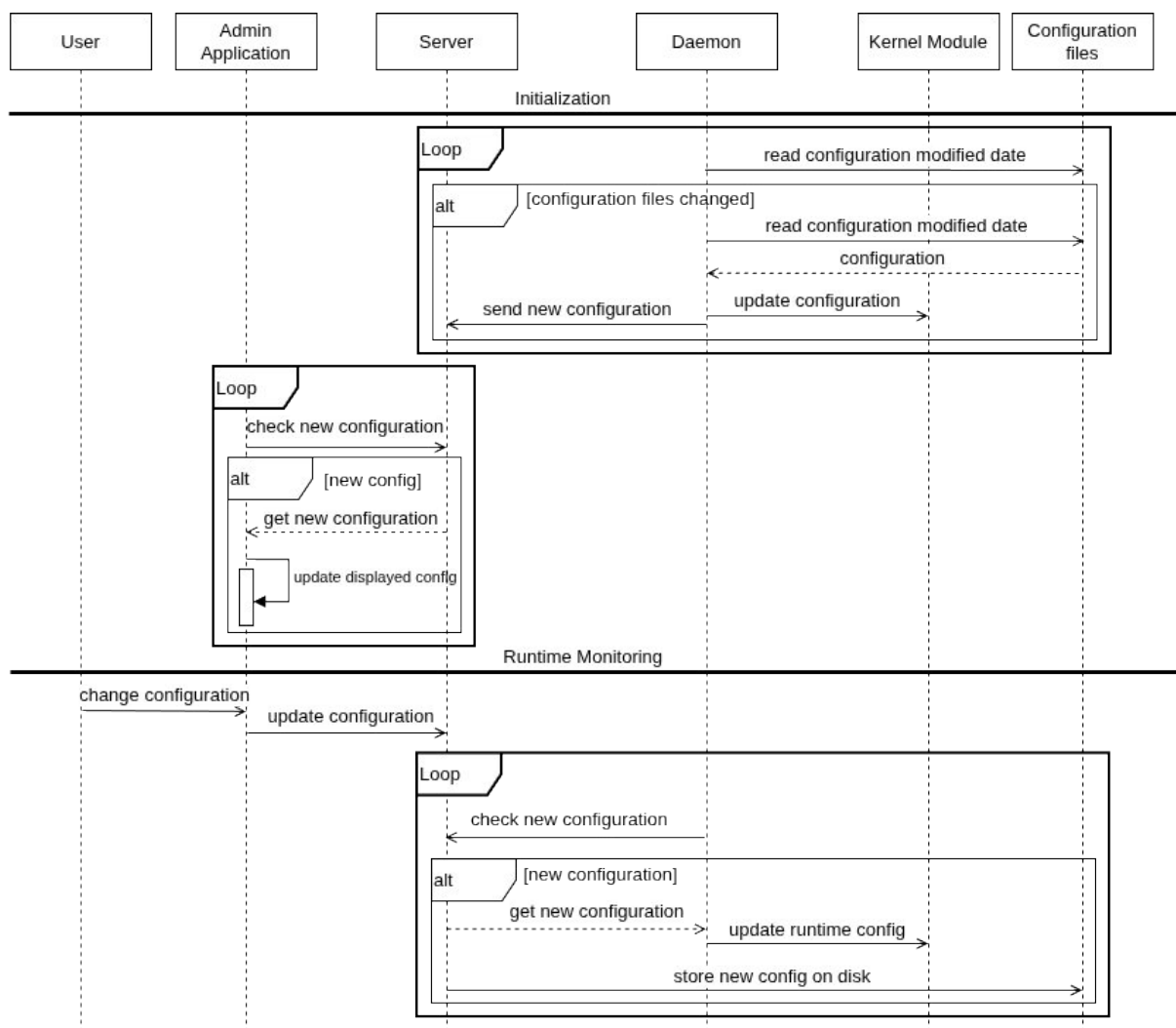


Imagine 1. Diagramă de componente

Pentru a evidenția procesul de transmitere a datelor în cadrul aplicației, au fost realizate două diagrame secvențiale. Prima diagramă ilustrează traseul parcurs de informațiile asociate pachetelor interceptate de *componenta de filtrare* [Imagine 2. Diagramă secvențială - informații pachete], iar cea de-a doua prezintă fluxul de actualizare și propagare a configurației echipamentelor [Imagine 3. Diagramă secvențială - configurații].



Imagine 2. Diagramă secvențială - informații pachete



Imagine 3. Diagramă secvențială - configurații

Experiența utilizatorului

Pentru asigurarea rulării corecte a aplicației, se recomandă utilizarea pe echipament a unei distribuții *Linux* (pentru testare s-a folosit *Debian*) cu versiunea de kernel 6.8.0-51.

Pentru utilizarea aplicației pe dispozitivele vizate pentru monitorizare, sunt necesari patru pași:

1. **Crearea fișierelor de configurare**, proces realizat prin definirea fișierului de configurare și de definire a conexiunilor dorite. Pentru configurare, aplicația va căuta fișierul „*mymodule.conf*”, în locația „*/etc/*”. În acest fișier [Fragment de cod 4. *mymodule.conf*] va fi definită adresa de IP locală a serverului căruia i se vor trimite informațiile referitoare la pachete, portul pe care acesta va trebui să îl folosească în comunicarea cu serverul, locația fișierului ce va fi utilizat în definirea conexiunilor permise și modul de funcționare a aplicației pe dispozitivul suport.

```
# Pentru a comenta o linie, se poate folosi
# caracterul „#”

# exemplu de adresă - 192.168.0.112
server IP <adresa ip a serverului>

# portul trebuie regăsit în valoare numerică
server port <port>

# se dorește locația globală a fișierului
allowed file <locația fișierului de definiții>

<mod>
# Modul de rulare este procesat fără diferențiere între
# majuscule și minuscule.
# se acceptă - listening, Listening, LISTENING, LIstEnInG
```

Fragment de cod 4. mymodule.conf

2. **Compilarea modulului kernel** se realizează prin rularea comenzii „*make*”. Această comandă utilizează fișierul de configurare *Makefile* [Fragment de cod 5. *Makefile*]:

```
obj-m += kernel_module.o

KDIR := /lib/modules/$(shell uname -r)/build
PWD := $(shell pwd)

all:
    $(MAKE) -C $(KDIR) M=$(PWD) modules

clean:
    $(MAKE) -C $(KDIR) M=$(PWD) clean
```

Fragment de cod 5. Makefile

3. **Adăugarea modului în spațiul kernel** realizată prin rularea comenzii „*sudo insmod kernel_module.ko*”.
4. **Rularea daemonului** se face prin executarea binarului cu privilegii elevate:
„*sudo ./daemon*”.

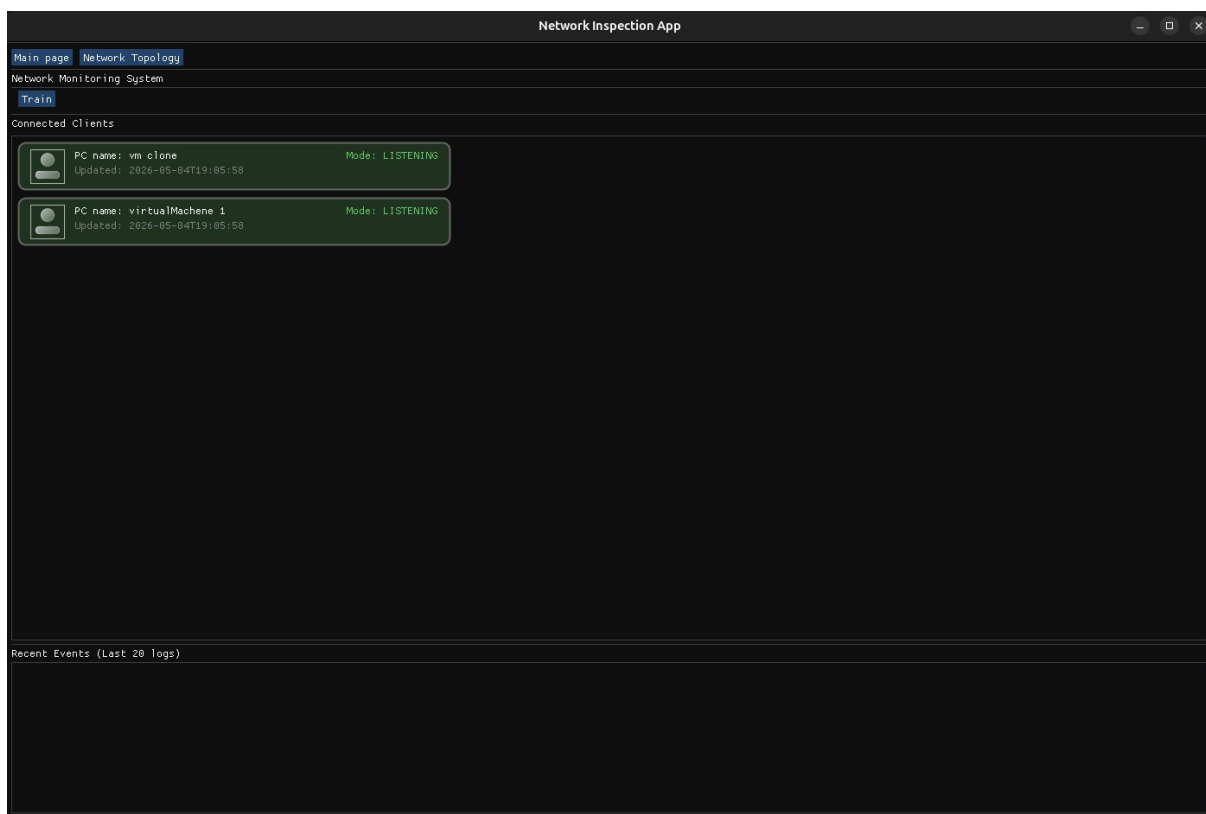
Aplicația desktop va porni din **fereastra principală** [Imagine 4. pagina principală]. Aceasta oferă posibilitatea de a vizualiza dispozitivele ce au modulul kernel activ și care sunt în comunicare cu serverul.

În partea de sus a interfeței grafice există un meniu de mutare între paginile aplicației. Primul buton face trimiterea către pagina principală, iar cel de al doilea buton, către pagina de vizualizare grafică a conexiunilor începute în rețeaua locală – inspecția *Topologiei Rețelei*. Acest meniu va rămâne prezent indiferent de pagina curentă a aplicației.

În secțiunea principală a paginii se va regăsi o listă cu toate dispozitivele curente. Fiecare dispozitiv poate fi însoțit de o iconiță modificabilă reprezentativă componentei, un nume oferit acestuia, informații legat de ultima dată de comunicare cu acest dispozitiv, și modul curent de rulare a acestuia. Culoarea de fundal a fiecărui element este reprezentativă modului curent de rulare:

1. Roșu – REACTIVE
2. Verde – LISTENING
3. Albastru - MONITORING
4. Gri – DISABLED

Partea inferioară a paginii este desemnată informării legat de evenimentele recent petrecute în rețea. Aceste evenimente reprezintă conexiuni ce au fost raportate de dispozitive, ce sunt afișate în ordine descrescătoare a datei de raportare.

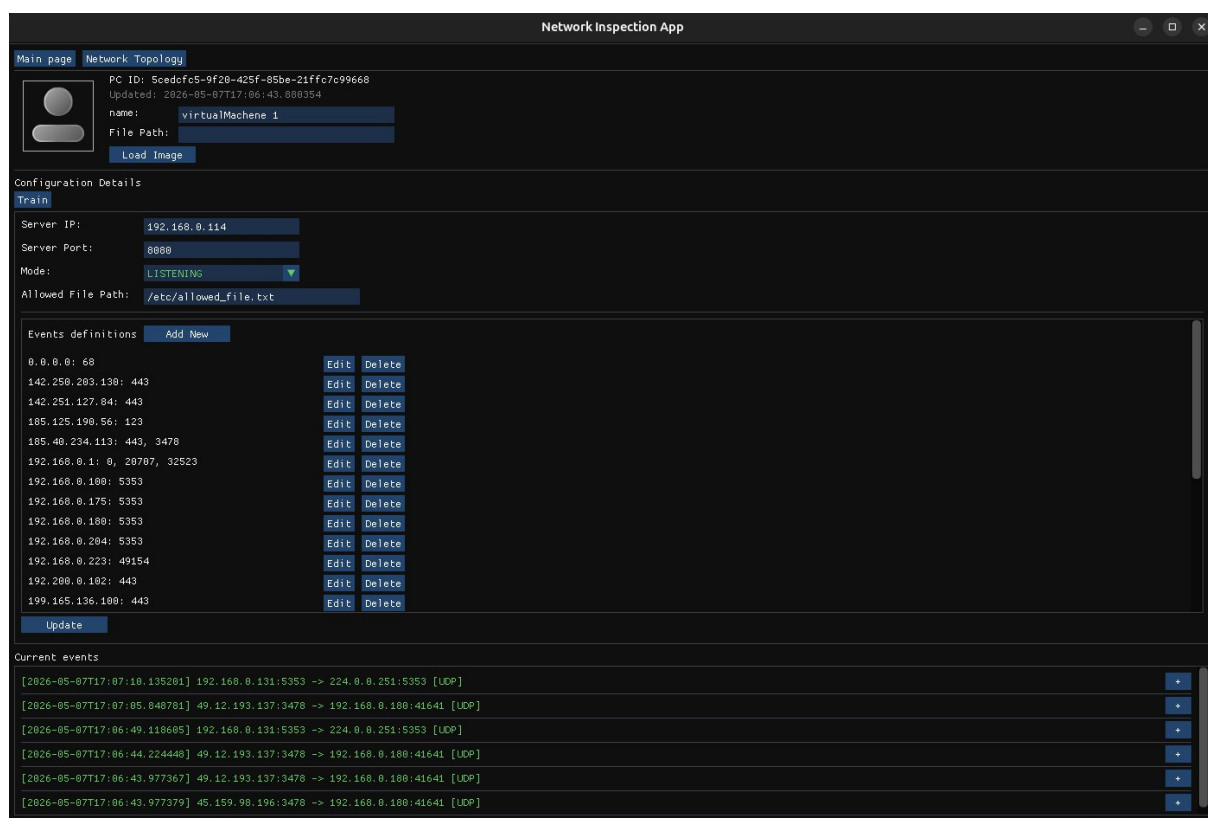


Imagine 4. pagina principală

În această pagină există de asemenea opțiunea de antrenare automată (ce se activează prin apăsarea butonului „Train”). În urma activării, echipamentele din rețeaua locală sunt trecute în modul de funcționare *MONITORING*. În acest mod conexiunile raportate sunt adăugate automat în lista de conexiuni permise. Acest mod este util pentru stabilirea automată a relațiilor dintre dispozitive, permițând construirea progresivă a unei liste de conexiuni permise pe baza traficului observat în rețea. Această antrenare se dezactivează prin reapăsarea aceluiași buton.

Selectând pictograma unui echipament, utilizatorul este direcționat către pagina privată a acestuia [Imagine 5. pagină echipament], unde poate vizualiza și edita atât setările de configurare, cât și informațiile asociate. În prima secțiune se regăsesc informații generale: identificatorul unic al echipamentului generat de aplicație, data de la ultima comunicare cu acest echipament, numele și iconița oferită dispozitivului. În această secțiune pot fi modificate numele și iconița echipamentului. Mai jos se regăsesc informațiile de funcționare ale dispozitivului. Primul bloc afișează configurația fișierului „*mymodule.conf*”, care include adresa IP a componentei centrale, portul de comunicare, modul de execuție curent al echipamentului și locația fișierului cu adresele permise. În următorul bloc se regăsesc adresele conexiunilor permise, sub formă de listă editabilă.

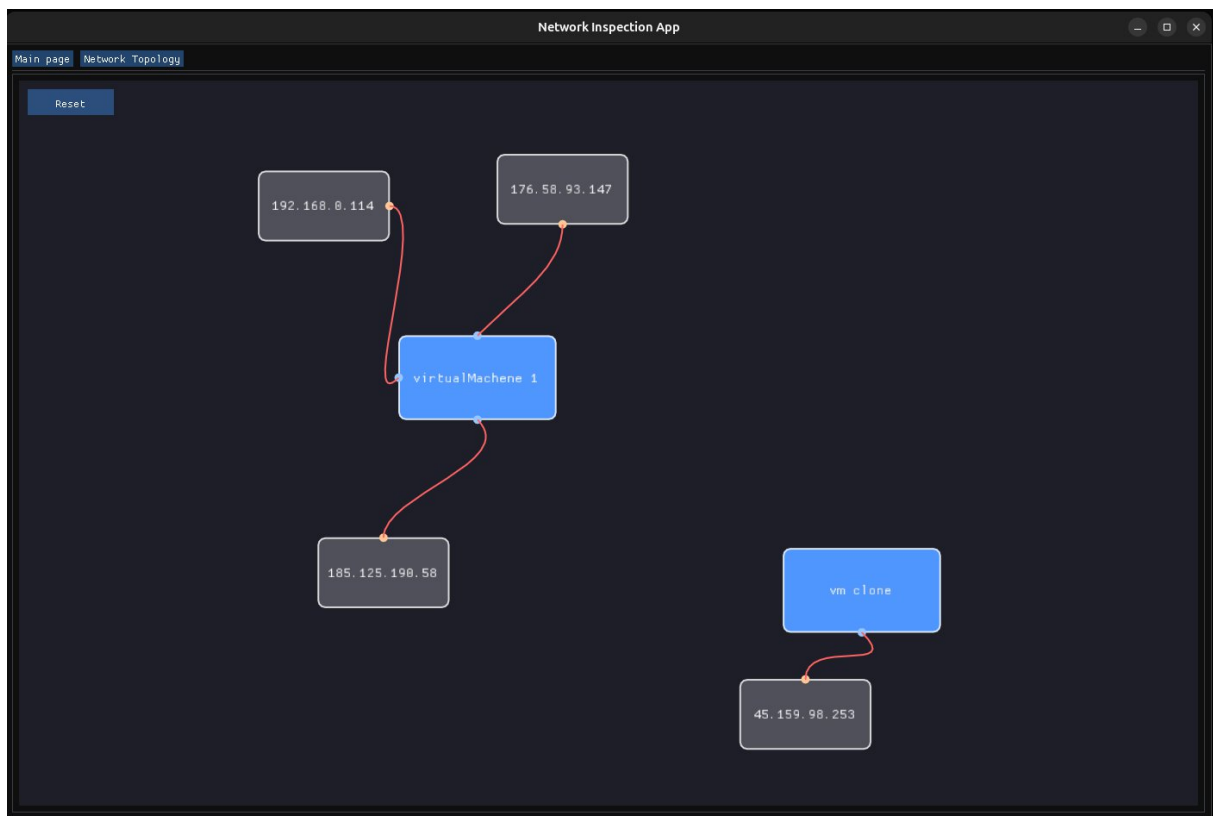
Pagina se încheie cu o secțiune asemănătoare paginii principale, având ca și conținut un scurt istoric al celor mai recente conexiuni, dar care este limitată la informații în conexiune cu echipamentul.



Imagine 5. pagină echipament

În urma aplicării unor modificări, informația este salvată și aplicată prin apăsarea butonului „Update”.

Pagina **Topologiei rețelei** [Imagine 6. Topologia rețelei] permite vizualizarea grafică de ansamblu a conexiunilor din rețeaua locală. În această pagină, fiecare echipament este reprezentat printr-un nod distinct, iar legăturile dintre acestea ilustrează conexiunile active din rețea. Nodurile afișează fie adresa IP a dispozitivului (în cazul în care acesta nu este un echipament controlat), fie numele acestuia, oferind o imagine clară asupra structurii rețelei. Pagina include și un buton de resetare a vizualizării, util în situațiile în care reprezentarea devine aglomerată din cauza numărului mare de elemente.



Imagine 6. Topologia rețelei

Concluzii și planuri de viitor

Conceptul aplicației prezentate reprezintă o soluție funcțională de securizare a echipamentelor în medii industriale, unde relațiile între componentele ce dețin drepturi de comunicare externă trebuie controlate și pot fi definite. Aplicația pune baza unui mecanism simplificat de control al accesului pe bază de politici, prin definirea explicită a conexiunilor permise între echipamente.

Prin monitorizarea traficului în rețea și aplicarea regulilor de configurare, sistemul permite limitarea comunicării neautorizate și oferă control asupra interacțiunilor dintre componentele infrastructurii. Abordarea utilizată facilitează identificarea rapidă a conexiunilor noi sau neprevăzute și reduce suprafața de atac a dispozitivelor monitorizate.

În cadrul dezvoltării ulterioare, aplicația poate fi extinsă prin adăugarea unor mecanisme de învățare a tiparelor considerate normale în rețea și identificarea deviațiilor de la aceste tipare. Reacționarea automată pentru un număr de cazuri poate reprezenta de asemenea o funcționalitate utilă aplicației: în cazul unui dispozitiv configurat să accepte comunicații

exclusiv de la echipamente definite explicit — mod de funcționare REACTIVE — detectarea unui număr anormal de pachete provenite de la o sursă acceptată poate indica o activitate suspectă. Aplicația poate reacționa prin notificări către administrator sau blocarea emițătorului, în funcție de configurare.

Bibliografie

- [1] Adeeb Alhomoud, R. Munir, J. P. Disso, I. Awan și A. Al-Dhelaan, 2011, „Performance Evaluation Study of Intrusion Detection Systems”, *Procedia Computer Science*, vol. 5, pp. 173–180.
- [2] T. K. Das, S. Adepu și J. Zhou, 2020, „Anomaly detection in Industrial Control Systems using Logical Analysis of Data”, *Computers & Security*, vol. 96, art. 101935.
- [3] M. Elnour, M. Noorizadeh, M. Shakerpour, N. Meskin, K. Khan și R. Jain, 2023, „A Machine Learning Based Framework for Real-Time Detection and Mitigation of Sensor False Data Injection Cyber-Physical Attacks in Industrial Control Systems”, *IEEE Access*, vol. 11, pp. 86977–86998, doi: 10.1109/ACCESS.2023.3303015.
- [4] J. Kim, J.-H. Yun și H. C. Kim, 2019, „Anomaly Detection for Industrial Control Systems Using Sequence-to-Sequence Neural Networks”, arXiv preprint arXiv:1911.04831.
- [5] L. Wang, 2026, „Clustering-Enhanced Domain Adaptation for Cross-Domain Intrusion Detection in Industrial Control Systems”, arXiv preprint arXiv:2604.12183.
- [6] The Linux Documentation Project, „Guides”, disponibil la: <https://tldp.org/guides.html> accesat la: 15 mai 2026.
- [7] Documentație Graphviz, disponibil la: <https://graphviz.org/> accesat la: 17 mai 2026.
- [8] Documentație Netfilter, disponibil la: <https://www.netfilter.org/> accesat la: 17 mai 2026
- [9] Raik Niemann, Udo Pfingst, Richard Göbel, 2015, „*Performance Evaluation of netfilter: A Study on the Performance Loss When Using netfilter as a Firewall*”, arXiv preprint arXiv:1502.05487
- [10] M. Nawrocki, T. C. Schmidt și M. Wählich, 2020, „Uncovering Vulnerable Industrial Control Systems from the Internet Core”, arXiv preprint arXiv:1901.04411.